Task 1: Watch the Terraform-06 video.
Task 2: Execute the script shown in the video.

1. Objective

To create an AWS EC2 instance using Terraform modules by separating the reusable resource configuration into a child module and calling it from the root module.

2. Why do we use Terraform Modules?

Terraform modules help us organize infrastructure code into reusable components instead of writing all resources in one file.

Real-time example: In a company, we may need to create web servers for Dev, Staging, and Production. Instead of writing the EC2 configuration repeatedly, we can create one reusable EC2 module and pass different values to it.

Benefits:

- Reusability: Use the same module for multiple servers.
- Maintainability: Update the configuration in one place.
- Organization: Separate root configuration from resource definitions.
- Consistency: Create resources using the same standard configuration.
- Scalability: Reuse modules as infrastructure grows.

3. Steps performed

1. Created a project folder named `Task-02-Modules`.
2. Created the root module configuration in `main.tf`.
3. Created a child module folder named `modules/ec2`.
4. Defined the EC2 resource in the child module.
5. Created `variables.tf` to accept the AMI, instance type, and instance name.
6. Configured the root module to call the child module using the `source` argument.
7. Executed `terraform init` to initialize Terraform and download the required AWS provider.
8. Executed `terraform validate` to check the configuration.
9. Executed `terraform plan` to preview the infrastructure changes.
10. Verified the AMI using the AWS CLI.
11. Executed `terraform apply` and confirmed the creation by typing `yes`.

12. Verified the EC2 instance state using the AWS CLI.

4. Code used

The following is the equivalent configuration for the module structure used in this task.

Folder structure

```
Task-02-Modules/
│
├── main.tf
├── variables.tf
└── modules/
    └── ec2/
        ├── main.tf
        └── variables.tf
```

File 1: Root `main.tf`

```
provider "aws" {
  region = "us-east-1"
}

module "dev_webserver" {
  source        = "./modules/ec2"
  ami           = var.ami
  instance_type = var.instance_type
  name          = var.name
}
```

Explanation:

- `provider`: Configures AWS as the cloud provider.
- `module "dev_webserver"`: Calls the child module.
- `source`: Specifies the child module directory.
- The remaining arguments pass variable values to the child module.

File 2: Root `variables.tf`

```
variable "ami" {
  type    = string
  default = "ami-02f3f602d23f1659d"
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}

variable "name" {
  type    = string
  default = "module-webserver"
}
```

File 3: `modules/ec2/main.tf`

```
resource "aws_instance" "webserver" {
  ami           = var.ami
  instance_type = var.instance_type

  tags = {
    Name = var.name
  }
}
```

Explanation:

- `aws_instance`: Defines the AWS EC2 resource.
- `ami`: Selects the Amazon Machine Image.
- `instance_type`: Specifies the EC2 instance size.
- `tags`: Assigns a name to the EC2 instance.

File 4: `modules/ec2/variables.tf`

```
variable "ami" {
  type = string
}
```

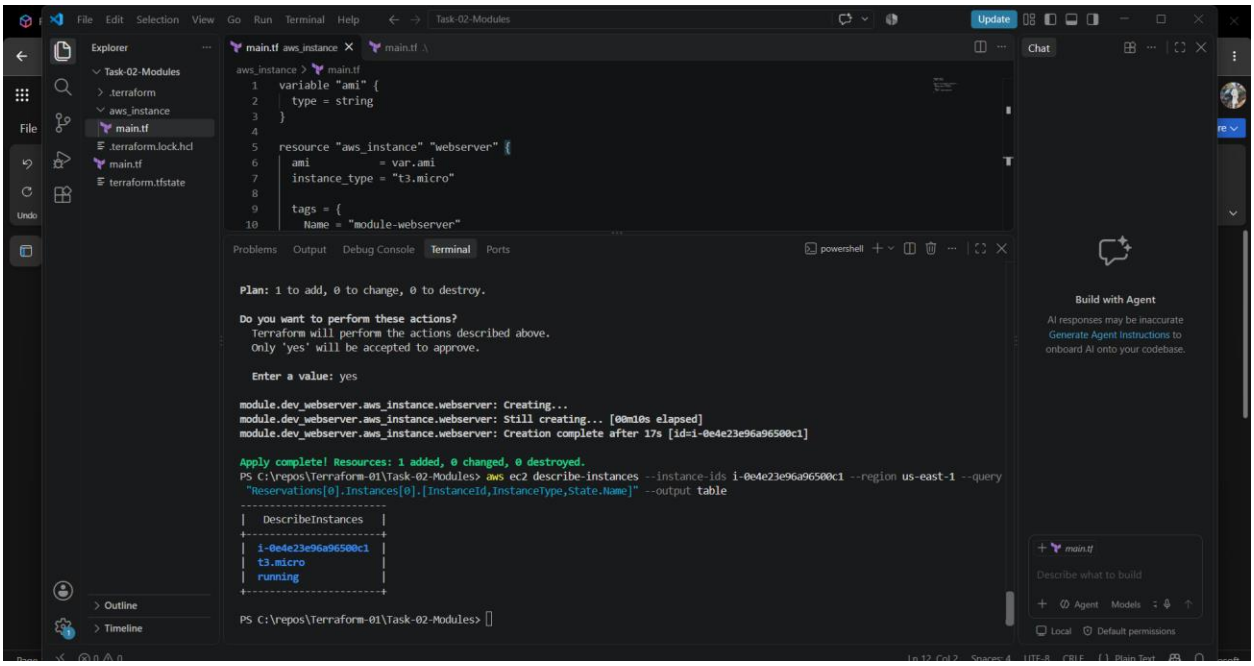
```
variable "instance_type" {
  type = string
}

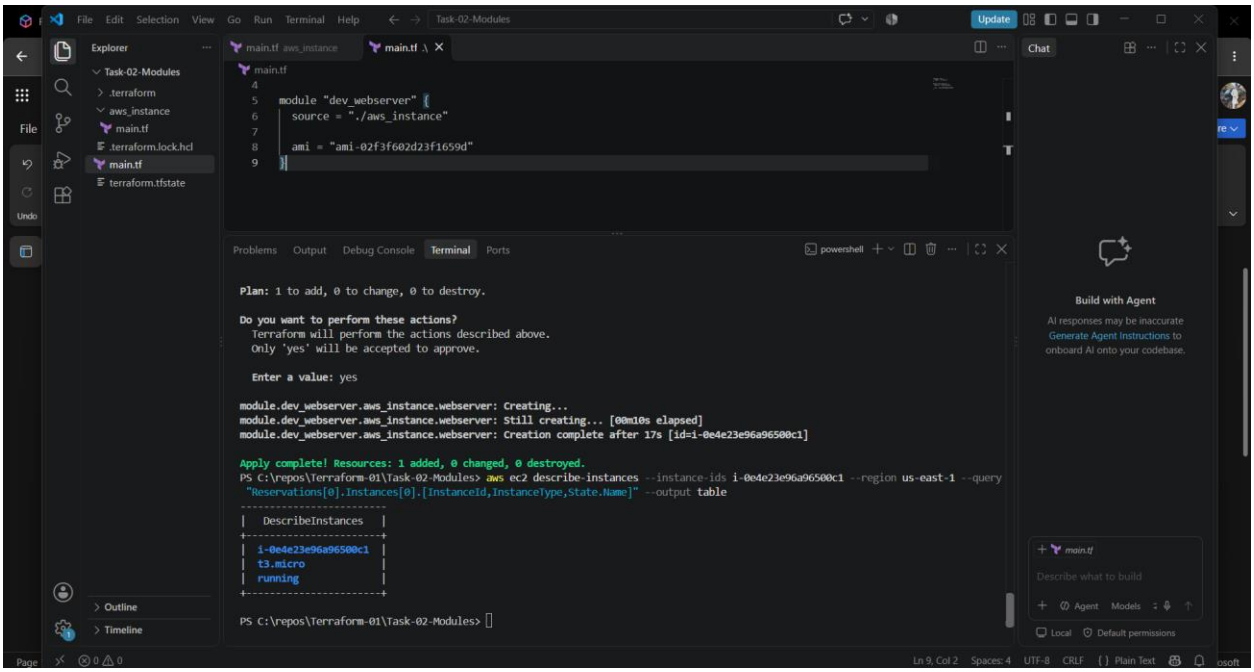
variable "name" {
  type = string
}
```

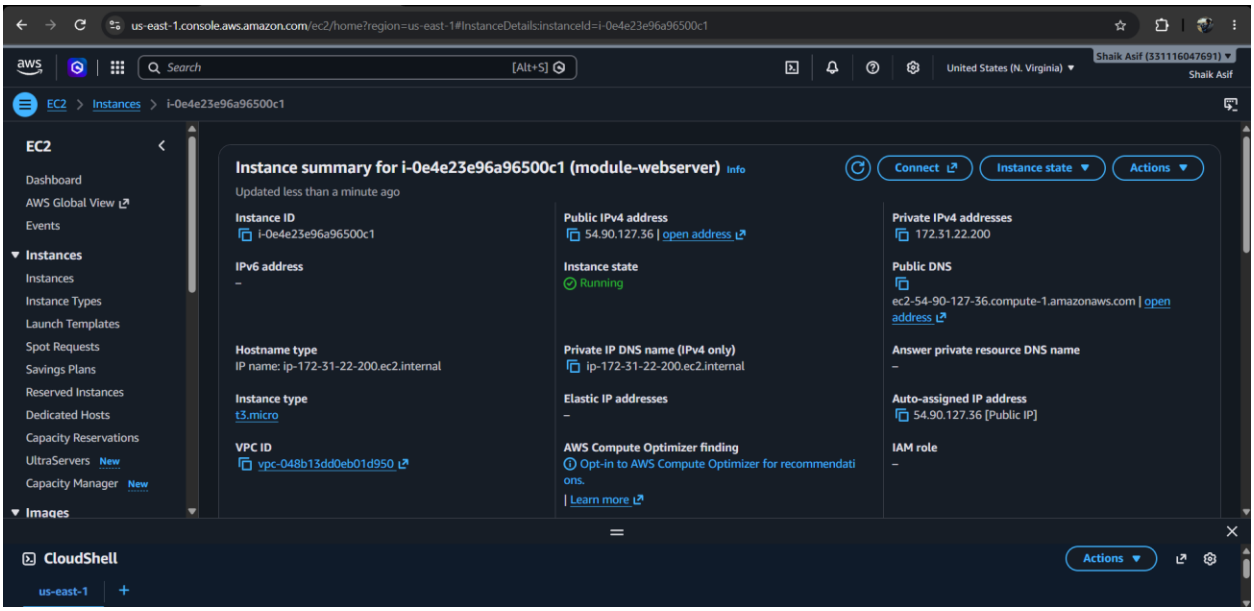
These variables receive their values from the root module.

5. Terraform commands used

Command	Purpose
`terraform init`	Initializes Terraform and providers.
`terraform validate`	Checks configuration syntax and structure.
`terraform plan`	Previews the changes Terraform intends to make.
`terraform apply`	Creates the infrastructure after confirmation.
`aws ec2 describe-images ...`	Verifies that the selected AMI is available.
`aws ec2 describe-instances ...`	Verifies the created EC2 instance.


File Edit Selection View Go Run Terminal Help
Task-02-Modules
Update
Explorer
Task-02-Modules
.terraform
aws_instance
main.tf
.terraform.lock.hcl
main.tf
terraform.tfstate
main.tf aws_instance
main.tf
aws_instance > main.tf
variable "ami" {
type = string
}
resource "aws_instance" "webserver" {
ami = var.ami
instance_type = "t3.micro"
tags = {
Name = "module-webserver"
Problems Output Debug Console Terminal Ports
powershell
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
module.dev_webserver.aws_instance.webserver: Creating...
module.dev_webserver.aws_instance.webserver: Still creating... [00m10s elapsed]
module.dev_webserver.aws_instance.webserver: Creation complete after 17s [id=i-0e4e23e96a96500c1]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\repos\Terraform-01\Task-02-Modules> aws ec2 describe-instances --instance-ids i-0e4e23e96a96500c1 --region us-east-1 --query
"Reservations[0].Instances[0].[InstanceId,InstanceType,State.Name]" --output table
DescribeInstances
i-0e4e23e96a96500c1
t3.micro
running
PS C:\repos\Terraform-01\Task-02-Modules>
Outline
Timeline
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to
onboard AI onto your codebase.
main.tf
Describe what to build
Agent Models
Local Default permissions


File Edit Selection View Go Run Terminal Help
Task-02-Modules
Update
Explorer
Task-02-Modules
.terraform
aws_instance
main.tf
.terraform.lock.hcl
main.tf
terraform.tfstate
main.tf aws_instance
main.tf
main.tf
module "dev_webserver" {
source = "./aws_instance"
ami = "ami-02f3f602d23f1659d"
}
Problems Output Debug Console Terminal Ports
powershell
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
module.dev_webserver.aws_instance.webserver: Creating...
module.dev_webserver.aws_instance.webserver: Still creating... [00m10s elapsed]
module.dev_webserver.aws_instance.webserver: Creation complete after 17s [id=i-0e4e23e96a96500c1]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\repos\Terraform-01\Task-02-Modules> aws ec2 describe-instances --instance-ids i-0e4e23e96a96500c1 --region us-east-1 --query
"Reservations[0].Instances[0].[InstanceId,InstanceType,State.Name]" --output table
DescribeInstances
i-0e4e23e96a96500c1
t3.micro
running
PS C:\repos\Terraform-01\Task-02-Modules>
Outline
Timeline
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to
onboard AI onto your codebase.
main.tf
Describe what to build
Agent Models
Local Default permissions



6. Observation

The Terraform plan displayed:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Terraform successfully created the EC2 instance through the child module.

Property	Result
Module name	`dev_webserver`
AWS resource	EC2 instance
Instance type	`t3.micro`
Region	`us-east-1`
Operating system image	Amazon Linux 2023

Instance ID	`i-0e4e23e96a96500c1`
Final state	`running`

The AWS CLI verification confirmed that the instance was running.

7. Conclusion

I successfully provisioned an AWS EC2 instance using Terraform modules. I learned how to create a child module, pass input variables from the root module, initialize the configuration, review the execution plan, and provision and verify the resource.

Terraform modules make infrastructure code reusable, organized, maintainable, and consistent across multiple environments.

Terraform Workspaces

1. Objective

To create and manage multiple projects or environments using the same Terraform configuration while maintaining separate state files for each workspace.

2. Why do we use workspaces?

Terraform workspaces are useful when multiple projects or environments need similar infrastructure configurations.

For example, ProjectA and ProjectB can use the same EC2 resource configuration while maintaining separate Terraform states.

- Reuse the same configuration.
- Maintain separate state for each workspace.
- Switch between projects or environments.

- Reduce duplicate Terraform code.

3. Steps performed

1. Created the `Task-02-Workspaces` project directory.
2. Created `main.tf` with the AWS provider and EC2 resource.
3. Configured the AMI and EC2 instance type.
4. Initialized Terraform using `terraform init`.
5. Listed available workspaces using `terraform workspace list`.
6. Created the `Dev`, `Staging`, and `Prod` workspaces.
7. Switched to the `Dev` workspace and executed `terraform plan`.
8. Applied the configuration and created the Dev EC2 instance.
9. Switched to the `Staging` workspace, reviewed the plan, and created the Staging EC2 instance.
10. Prepared to provision the Prod environment.

4. Workspace commands

Command	Purpose
`terraform workspace new ProjectA`	Creates a new workspace named ProjectA.
`terraform workspace list`	Lists available workspaces.
`terraform workspace select ProjectA`	Switches to ProjectA.
`terraform workspace show`	Displays the current workspace.
`terraform workspace delete ProjectA`	Deletes a workspace if its state and resources are handled appropriately.

5. Code used

`main.tf`

This is the configuration used in your actual lab, with workspace-specific names and tags.

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "webserver" {
  ami           = "ami-02f3f602d23f1659d"
  instance_type = "t3.micro"

  tags = {
    Name        = "workspace-webserver-${terraform.workspace}"
    Environment = terraform.workspace
  }
}
```

Alternative: Your original AMI map example

Your original notes use a map to select an AMI based on the workspace name. For that approach, the variable map keys must match the actual workspace names.

`main.tf`

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "webserver" {
  ami           = var.ami[terraform.workspace]
  instance_type = "t3.micro"

  tags = {
    Name = "webserver-${terraform.workspace}"
```

```
  }
}
```

variables.tf

```
variable "ami" {
  type = map(string)

  default = {
    default = "ami-02f3f602d23f1659d"
    Dev     = "ami-02f3f602d23f1659d"
    Staging = "ami-02f3f602d23f1659d"
    Prod    = "ami-02f3f602d23f1659d"
  }
}
```

Important: If you use `var.ami[terraform.workspace]`, every workspace you use must have a matching key in the map. Otherwise, Terraform cannot find the AMI value for that workspace.

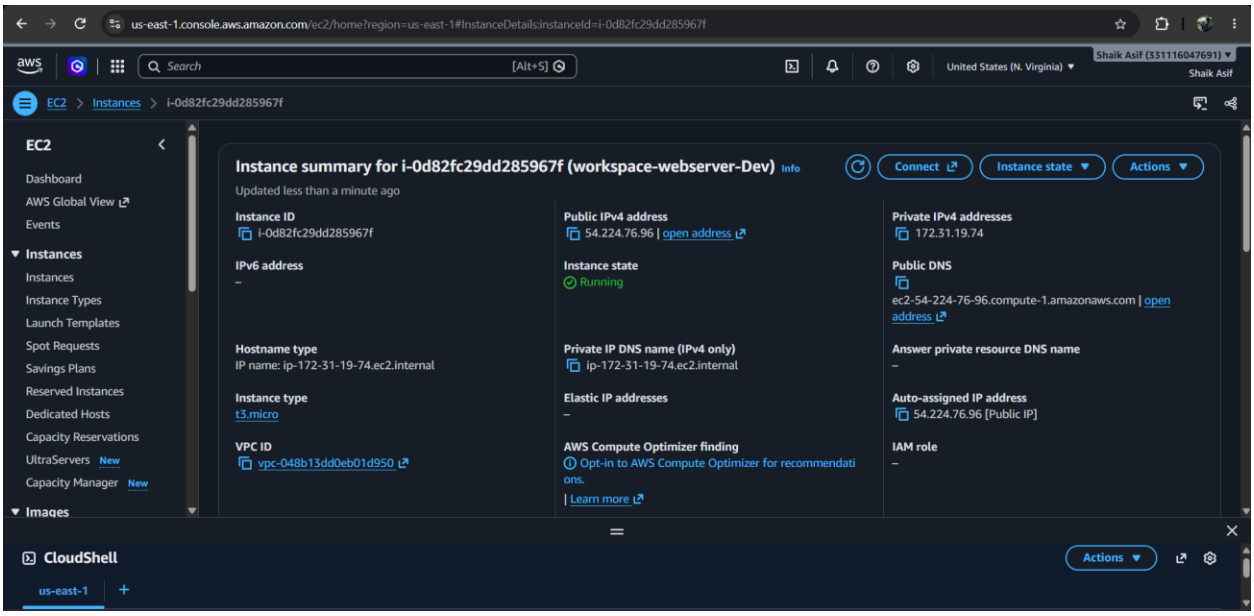
6. State file location

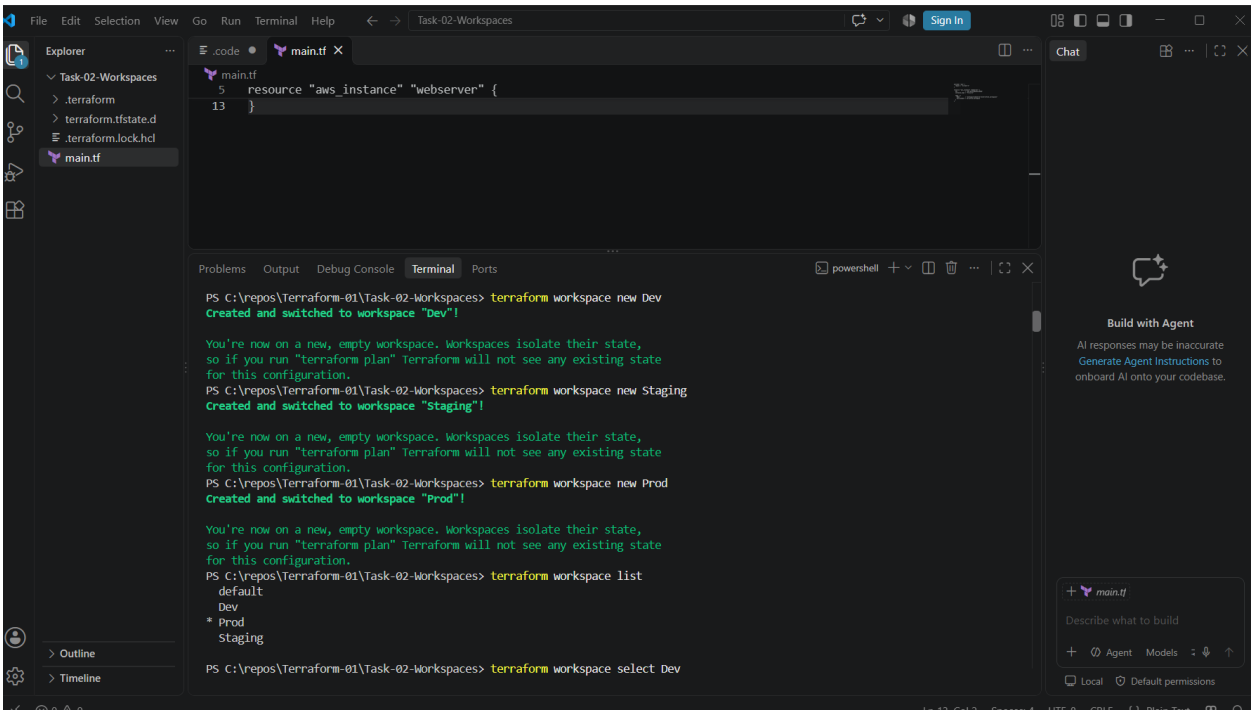
For Terraform's default local backend, workspace state files are stored under:

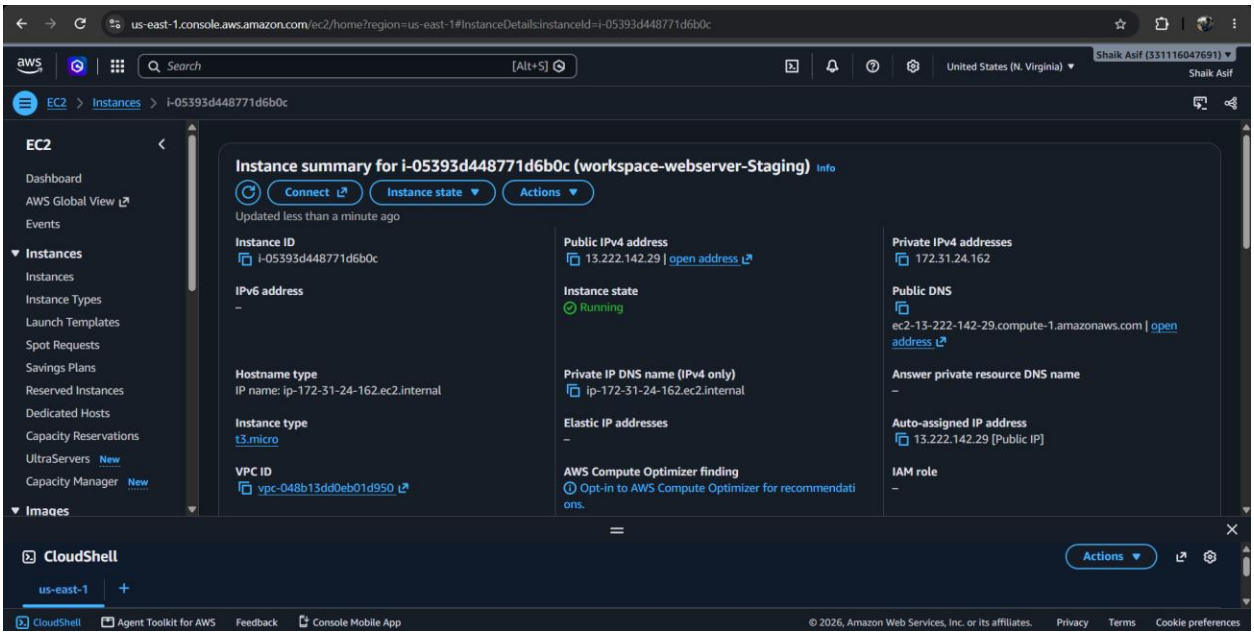
```
terraform.tfstate.d/
├── Dev/
│   └── terraform.tfstate
├── Staging/
│   └── terraform.tfstate
└── Prod/
    └── terraform.tfstate
```

The default workspace's state file is normally stored as `terraform.tfstate` in the root directory. Other workspaces use separate state files under `terraform.tfstate.d`.

quiring duplicate configuration files.


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0d82fc29dd285967f
Search
[Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
EC2 > Instances > i-0d82fc29dd285967f
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
Instance summary for i-0d82fc29dd285967f (workspace-webserver-Dev) Info
Updated less than a minute ago
Connect
Instance state
Actions
Instance ID
i-0d82fc29dd285967f
Public IPv4 address
54.224.76.96 | open address
Private IPv4 addresses
172.31.19.74
IPv6 address
–
Instance state
Running
Public DNS
ec2-54-224-76-96.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-172-31-19-74.ec2.internal
Private IP DNS name (IPv4 only)
ip-172-31-19-74.ec2.internal
Answer private resource DNS name
–
Instance type
t3.micro
Elastic IP addresses
–
Auto-assigned IP address
54.224.76.96 [Public IP]
VPC ID
vpc-048b13dd0eb01d950
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
| Learn more
IAM role
–
CloudShell
Actions
us-east-1


File Edit Selection View Go Run Terminal Help
Task-02-Workspaces
Sign In
Explorer
Task-02-Workspaces
.terraform
terraform.tfstate.d
.terraform.lock.hcl
main.tf
.code
main.tf
main.tf
5 resource "aws_instance" "webserver" {
13 }
Chat
Problems Output Debug Console Terminal Ports
powershell
PS C:\repos\Terraform-01\Task-02-Workspaces> terraform workspace new Dev
Created and switched to workspace "Dev"!
You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\repos\Terraform-01\Task-02-Workspaces> terraform workspace new Staging
Created and switched to workspace "Staging"!
You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\repos\Terraform-01\Task-02-Workspaces> terraform workspace new Prod
Created and switched to workspace "Prod"!
You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\repos\Terraform-01\Task-02-Workspaces> terraform workspace list
default
Dev
* Prod
Staging
PS C:\repos\Terraform-01\Task-02-Workspaces> terraform workspace select Dev
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
main.tf
Describe what to build
Agent
Models
Local
Default permissions
Outline
Timeline



7. Observation

- Terraform initialized successfully.
- Three workspaces — Dev, Staging, and Prod — were created.
- The Dev EC2 instance was created successfully: `i-0d82fc29dd285967f`.
- The Staging EC2 instance was created successfully: `i-05393d448771d6b0c`.
- The same EC2 configuration generated different resource tags based on the selected workspace.
- Prod workspace creation succeeded, but its EC2 instance creation has not been confirmed.

8. Conclusion

Terraform workspaces allow multiple environments or projects to reuse a single configuration while maintaining separate state files. By switching workspaces, Terraform manages each environment independently without req

Task 3: Provision EC2, S3, and VPC Using Terraform Modules

1. Objective

To provision an EC2 instance, an S3 bucket, and a VPC in AWS using separate reusable Terraform modules.

2. Steps Used

1. Created the project folder `Task-03-Modules`.
2. Created separate module folders for EC2, S3, and VPC.
3. Configured the AWS provider in the root `main.tf` file using region `us-east-1`.
4. Created the VPC resource in `modules/vpc/main.tf`.
5. Created the EC2 resource in `modules/ec2/main.tf`.
6. Created the S3 bucket resource in `modules/s3/main.tf`.
7. Called all three modules from the root `main.tf` file.
8. Executed `terraform init` to initialize Terraform and download the AWS provider.
9. Executed `terraform validate` to validate the configuration.
10. Executed `terraform plan` to review the resources to be created.
11. Executed `terraform apply` and entered `yes` to provision the resources.
12. Executed `terraform state list` to verify the resources.

3. Code Used

A. Root file — `main.tf`

```
provider "aws" {
  region = "us-east-1"
}

module "vpc" {
  source = "./modules/vpc"
}

module "ec2" {
  source = "./modules/ec2"
}
```

```
module "s3" {
  source = "./modules/s3"
}
```

B. VPC module — `modules/vpc/main.tf`

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-modules-vpc"
  }
}
```

C. EC2 module — `modules/ec2/main.tf`

```
resource "aws_instance" "web" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t3.micro"

  tags = {
    Name = "terraform-modules-ec2"
  }
}
```

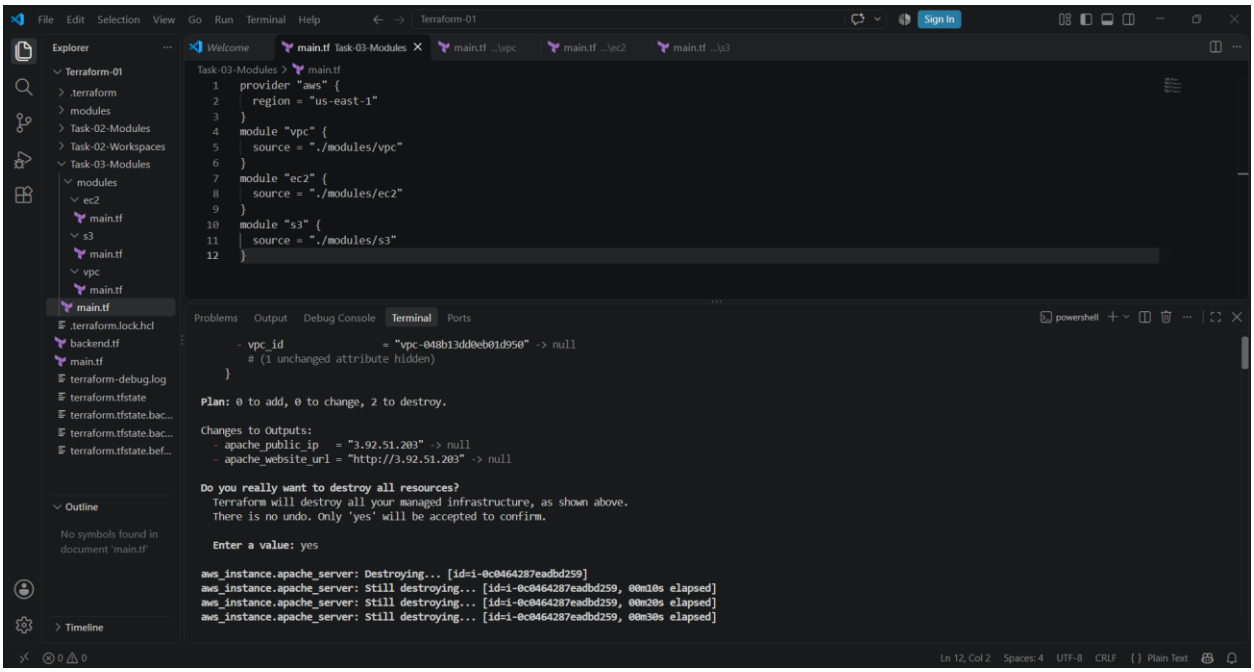
D. S3 module — `modules/s3/main.tf`

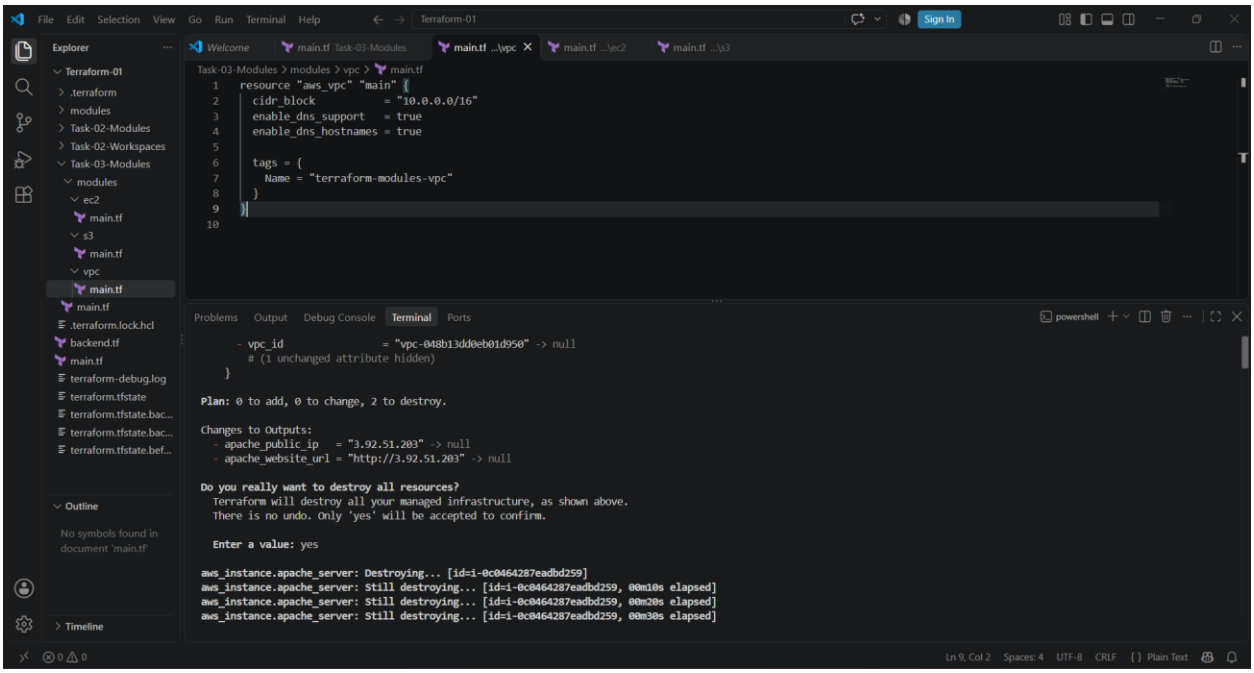
```
resource "aws_s3_bucket" "main" {
  bucket = "asif-terraform-modules-s3-2026"

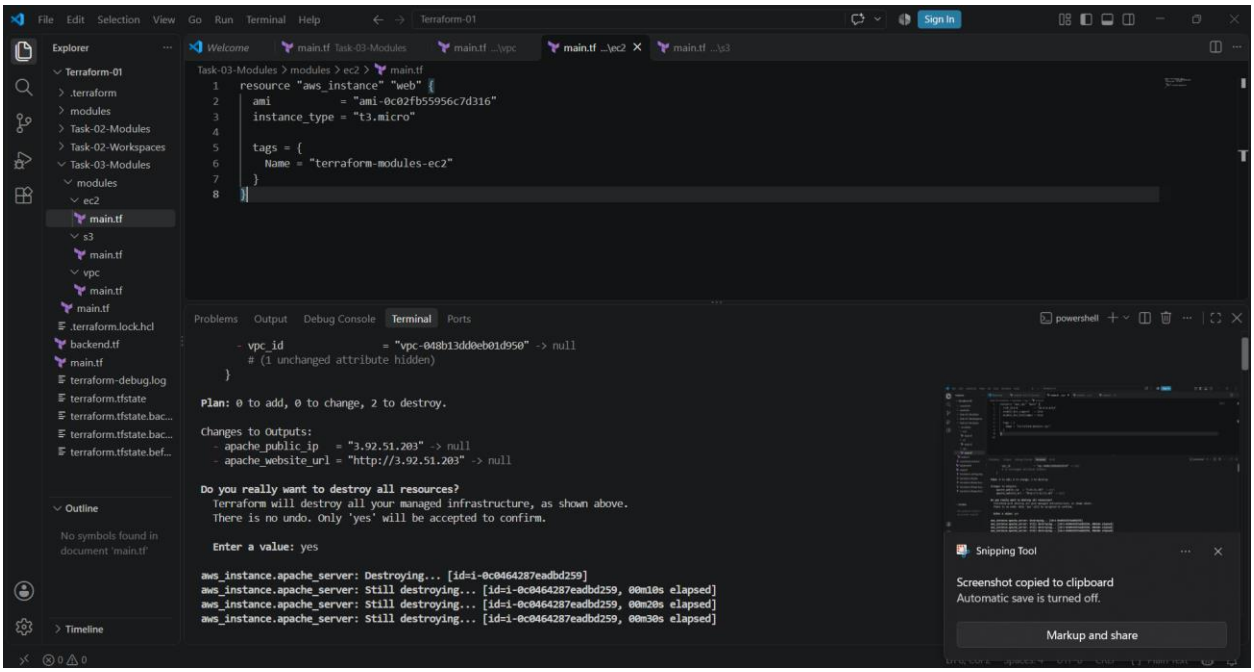
  tags = {
    Name = "terraform-modules-s3"
  }
}
```

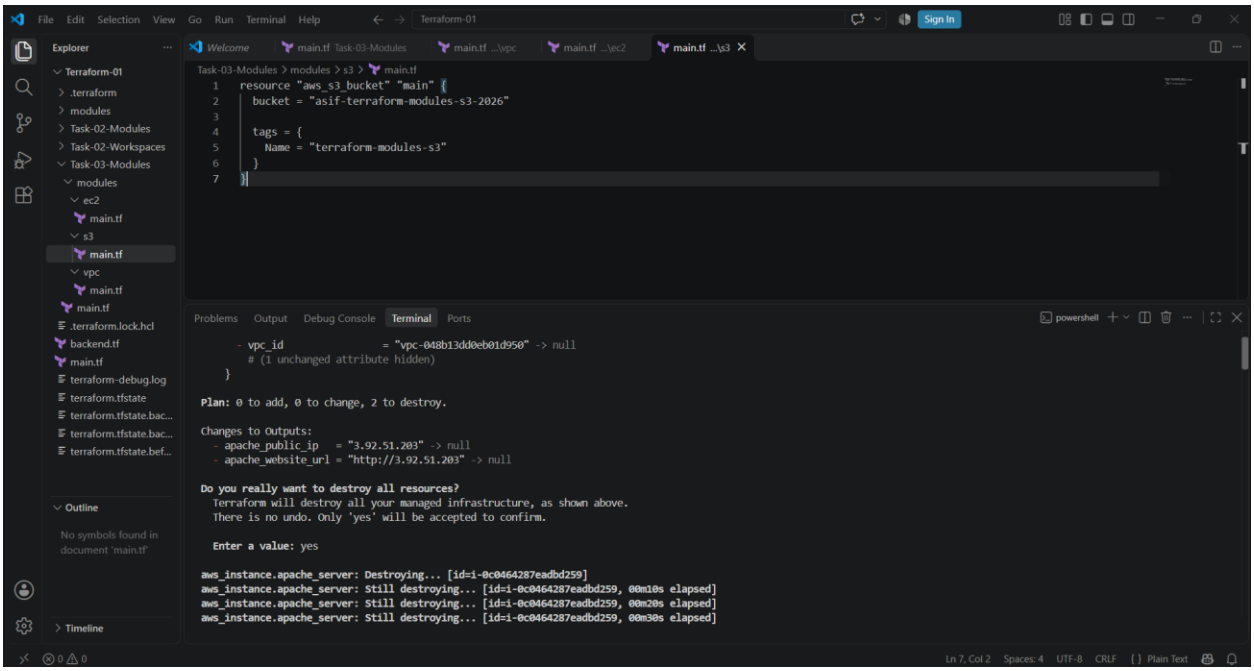
4. Why Do We Need Terraform Modules?

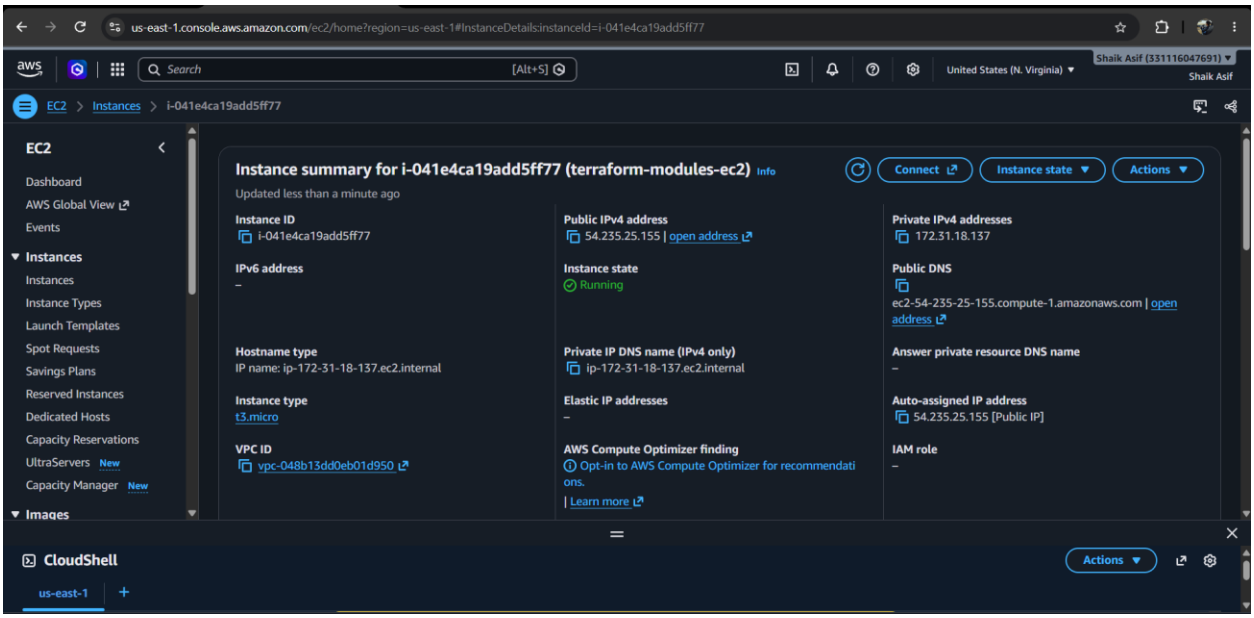
- Reusability: Reuse the same module across multiple projects.
- Organization: Keep EC2, S3, and VPC configurations in separate folders.
- Maintainability: Update one module without mixing all resource configurations.
- Consistency: Use the same resource configuration wherever the module is called.
- Scalability: Add or reuse modules as infrastructure requirements grow.

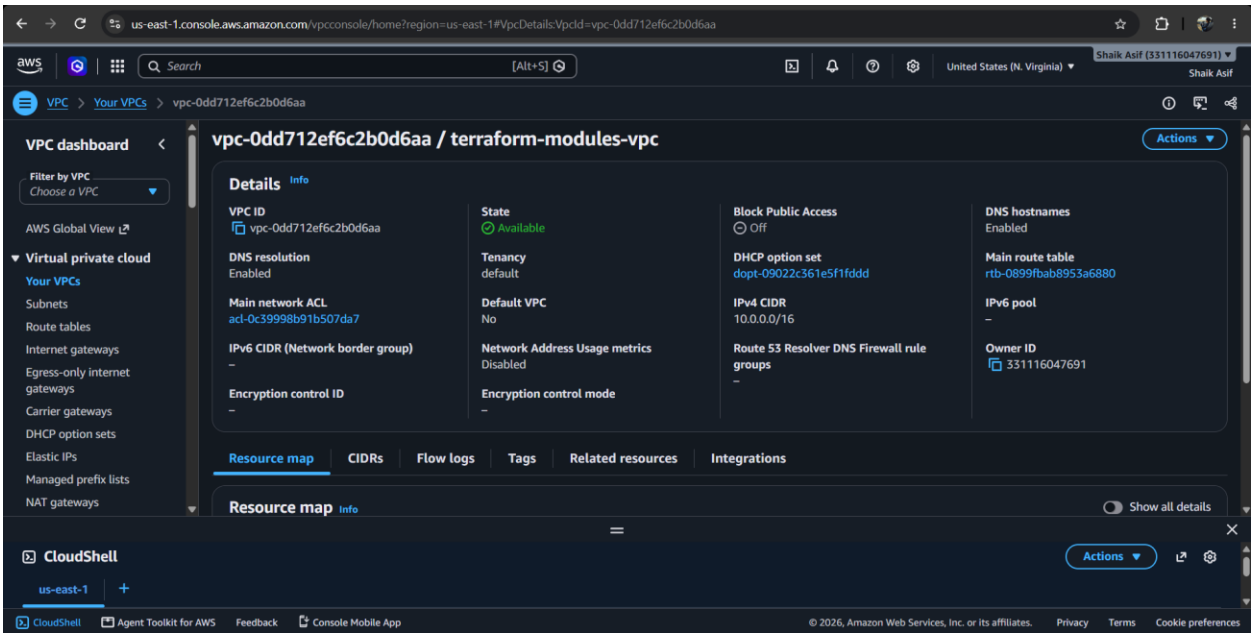


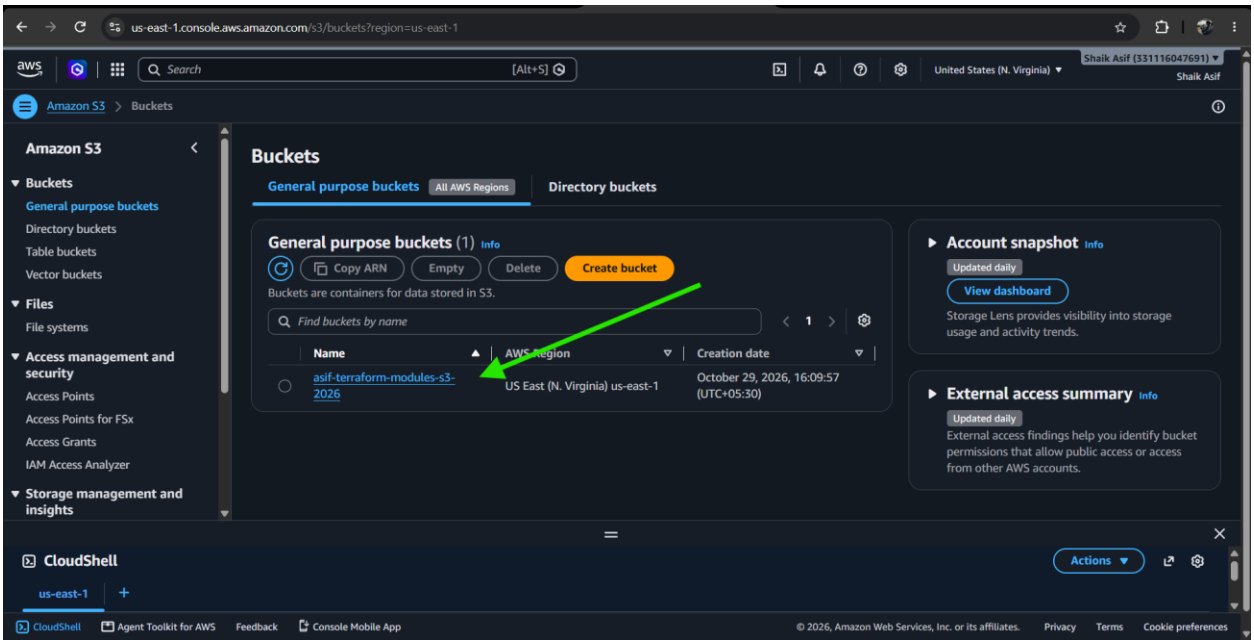

Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
main.tf
.terraform.lock.hcl
backend.tf
main.tf
terraform-debug.log
terraform.tfstate
terraform.tfstate.bac...
terraform.tfstate.bac...
terraform.tfstate.bef...
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
Task-03-Modules > modules > vpc > main.tf
resource "aws_vpc" "main" {
cidr_block = "10.0.0.0/16"
enable_dns_support = true
enable_dns_hostnames = true
tags = {
Name = "terraform-modules-vpc"
}
}
Problems
Output
Debug Console
Terminal
Ports
powershell
- vpc_id = "vpc-048b13dd0eb01d950" -> null
(1 unchanged attribute hidden)
}
Plan: 0 to add, 0 to change, 2 to destroy.
Changes to Outputs:
- apache_public_ip = "3.92.51.203" -> null
- apache_website_url = "http://3.92.51.203" -> null
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
Enter a value: yes
aws_instance.apache_server: Destroying... [id=i-0c0464287eadbd259]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m10s elapsed]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m20s elapsed]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m30s elapsed]
Ln 9, Col 2
Spaces: 4
UTF-8
CRLF
Plain Text

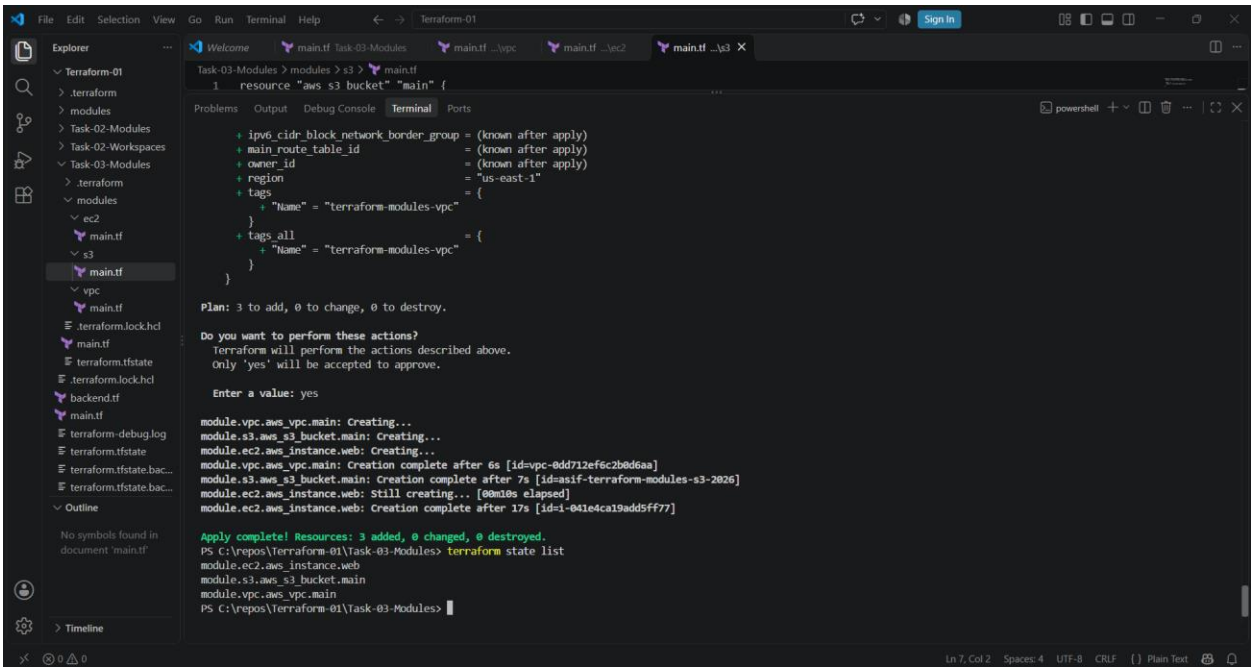

Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
main.tf
.terraform.lock.hcl
backend.tf
main.tf
terraform-debug.log
terraform.tfstate
terraform.tfstate.bac...
terraform.tfstate.bac...
terraform.tfstate.bef...
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
Task-03-Modules > modules > ec2 > main.tf
resource "aws_instance" "web" {
ami = "ami-0c02fb55956c7d316"
instance_type = "t3.micro"
tags = {
Name = "terraform-modules-ec2"
}
}
Problems
Output
Debug Console
Terminal
Ports
powershell
- vpc_id = "vpc-048b13dd0eb01d950" -> null
(1 unchanged attribute hidden)
}
Plan: 0 to add, 0 to change, 2 to destroy.
Changes to Outputs:
- apache_public_ip = "3.92.51.203" -> null
- apache_website_url = "http://3.92.51.203" -> null
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
Enter a value: yes
aws_instance.apache_server: Destroying... [id=i-0c0464287eadbd259]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m10s elapsed]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m20s elapsed]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m30s elapsed]
Snipping Tool
Screenshot copied to clipboard
Automatic save is turned off.
Markup and share


File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
main.tf
.terraform.lock.hcl
backend.tf
main.tf
terraform-debug.log
terraform.tfstate
terraform.tfstate.bac...
terraform.tfstate.bac...
terraform.tfstate.bef...
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
Task-03-Modules > modules > s3 > main.tf
1 resource "aws_s3_bucket" "main" {
2 bucket = "asif-terraform-modules-s3-2026"
3
4 tags = {
5 Name = "terraform-modules-s3"
6 }
7 }
Problems Output Debug Console Terminal Ports
powershell
- vpc_id = "vpc-048b13dd0eb01d950" -> null
(1 unchanged attribute hidden)
}
Plan: 0 to add, 0 to change, 2 to destroy.
Changes to Outputs:
- apache_public_ip = "3.92.51.203" -> null
- apache_website_url = "http://3.92.51.203" -> null
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
Enter a value: yes
aws_instance.apache_server: Destroying... [id=i-0c0464287eadbd259]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m10s elapsed]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m20s elapsed]
aws_instance.apache_server: Still destroying... [id=i-0c0464287eadbd259, 00m30s elapsed]
Ln 7, Col 2 Spaces: 4 UTF-8 CRLF { } Plain Text


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-041e4ca19add5ff77
Search [Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
EC2 > Instances > i-041e4ca19add5ff77
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
Instance summary for i-041e4ca19add5ff77 (terraform-modules-ec2) Info
Updated less than a minute ago
Connect
Instance state
Actions
Instance ID
i-041e4ca19add5ff77
Public IPv4 address
54.235.25.155 | open address
Private IPv4 addresses
172.31.18.137
IPv6 address
–
Instance state
Running
Public DNS
ec2-54-235-25-155.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-172-31-18-137.ec2.internal
Private IP DNS name (IPv4 only)
ip-172-31-18-137.ec2.internal
Answer private resource DNS name
–
Instance type
t3.micro
Elastic IP addresses
–
Auto-assigned IP address
54.235.25.155 [Public IP]
VPC ID
vpc-048b13dd0eb01d950
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more
IAM role
–
CloudShell
Actions
us-east-1


us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#VpcDetails:VpcId=vpc-0dd712ef6c2b0d6aa
Search
[Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
VPC > Your VPCs > vpc-0dd712ef6c2b0d6aa
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
vpc-0dd712ef6c2b0d6aa / terraform-modules-vpc
Actions
Details Info
VPC ID
vpc-0dd712ef6c2b0d6aa
State
Available
Block Public Access
Off
DNS hostnames
Enabled
DNS resolution
Enabled
Tenancy
default
DHCP option set
dopt-09022c361e5f1fddd
Main route table
rtb-0899fbab8953a6880
Main network ACL
acl-0c39998b91b507da7
Default VPC
No
IPv4 CIDR
10.0.0.0/16
IPv6 pool
–
IPv6 CIDR (Network border group)
–
Network Address Usage metrics
Disabled
Route 53 Resolver DNS Firewall rule groups
–
Owner ID
331116047691
Encryption control ID
–
Encryption control mode
–
Resource map
CIDRs
Flow logs
Tags
Related resources
Integrations
Resource map Info
Show all details
CloudShell
Actions
us-east-1
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


us-east-1.console.aws.amazon.com/s3/buckets?region=us-east-1
Search
[Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
Amazon S3 > Buckets
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Buckets
General purpose buckets
All AWS Regions
Directory buckets
General purpose buckets (1) Info
Copy ARN
Empty
Delete
Create bucket
Buckets are containers for data stored in S3.
Find buckets by name
Name
AWS Region
Creation date
asif-terraform-modules-s3-2026
US East (N. Virginia) us-east-1
October 29, 2026, 16:09:57 (UTC+05:30)
Account snapshot Info
Updated daily
View dashboard
Storage Lens provides visibility into storage usage and activity trends.
External access summary Info
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.
CloudShell
Actions
us-east-1
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



5. Observation

Terraform successfully created all three AWS resources.

Resource	Created resource ID/name
VPC	`vpc-0dd712ef6c2b0d6aa`
S3 bucket	`asif-terraform-modules-s3-2026`
EC2 instance	`i-041e4ca19add5ff77`

The `terraform state list` command confirmed:

```
module.ec2.aws_instance.web
module.s3.aws_s3_bucket.main
module.vpc.aws_vpc.main
```

Result: 3 resources added, 0 changed, 0 destroyed.

6. Conclusion

Successfully provisioned EC2, S3, and VPC resources in AWS using separate Terraform modules. Verified the configuration using `terraform validate`, reviewed the execution plan, applied the configuration, and confirmed the resources in Terraform state. This task demonstrated how Terraform modules improve infrastructure organization, reusability, and maintainability.

Task 4: Provision EC2 for Dev, Staging, and Prod using Terraform Workspaces

1. Objective

To create separate EC2 instances for three environments — Development, Staging, and Production — using Terraform workspaces and a single Terraform configuration file.

2. Code used

File: `C:\repos\Terraform-01\Task-04-Workspaces\main.tf`

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c02fb55956c7d316"
```

```
  instance_type = "t3.micro"

  tags = {
    Name        = "terraform-${terraform.workspace}"
    Environment = terraform.workspace
  }
}
```

3. Why do we need Terraform workspaces?

In real-time projects, applications usually have multiple environments:

- Dev: Developers test new features.
- Staging: The team tests changes before production.
- Prod: The live application used by customers.

Without workspaces, we might need separate Terraform configurations or directories for each environment. Workspaces allow us to use the same configuration with separate Terraform state files.

This helps manage environments independently and reduces code duplication.

4. Steps used

1. Created the folder `Task-04-Workspaces`.
2. Created the `main.tf` file with the AWS provider and EC2 resource.
3. Initialized Terraform:

```
terraform init
```

4. Created the Dev workspace:

```
terraform workspace new dev
terraform plan
terraform apply
```

5. Created the Staging workspace:

```
terraform workspace new staging
terraform plan
terraform apply
```

6. Created the Prod workspace:

```
terraform workspace new prod
terraform plan
terraform apply
```

7. Verified the available workspaces:

```
terraform workspace list
```

8. Checked the current workspace:

```
terraform workspace show
```

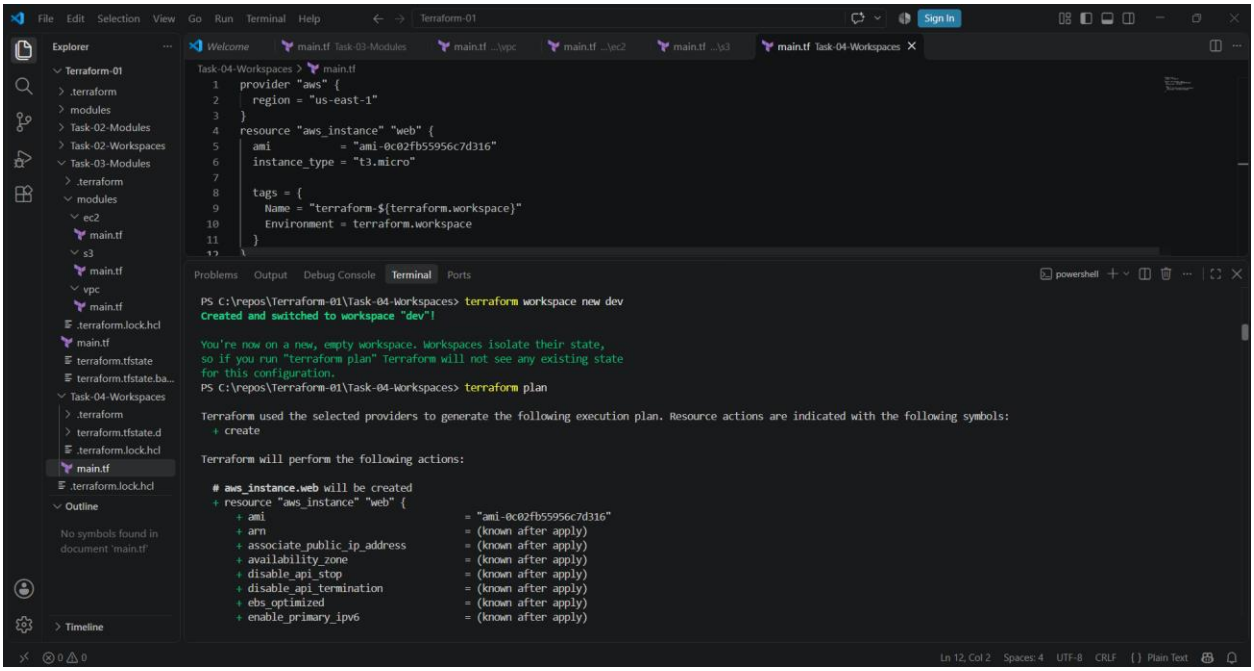
Output: `prod`

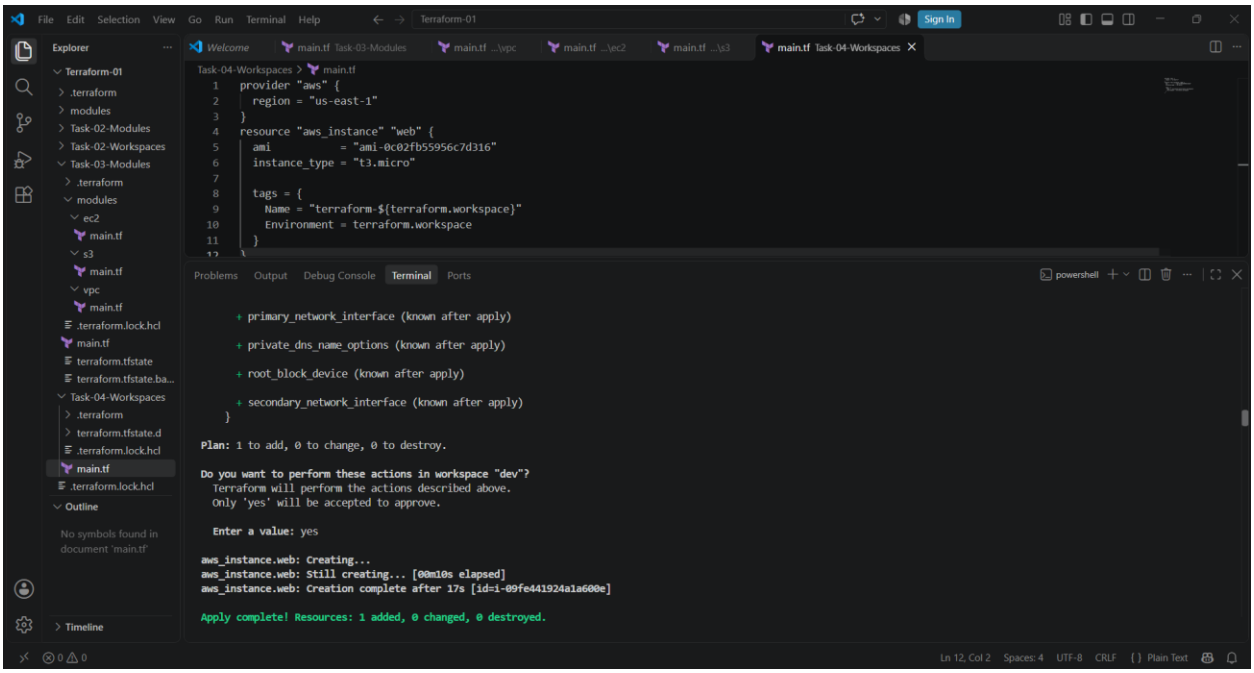
5. How the code works

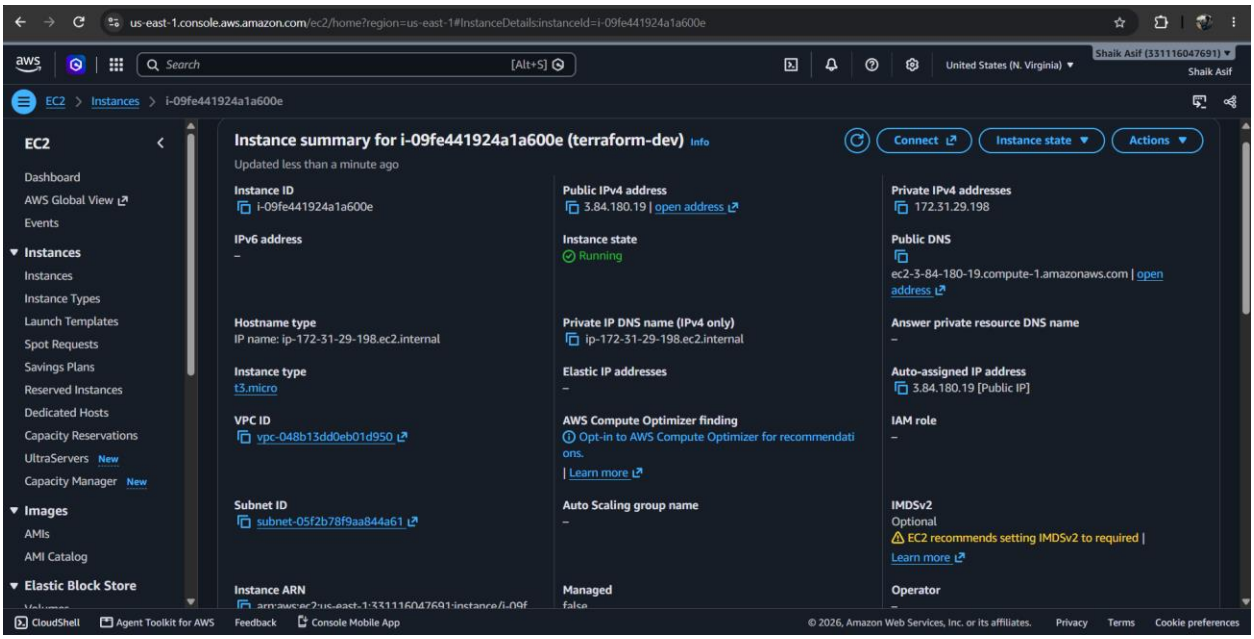
Code	Purpose
`provider "aws"`	Configures Terraform to use AWS.
`region = "us-east-1"`	Specifies the AWS region.
`resource "aws_instance" "web"`	Defines the EC2 instance to create.
`ami`	Specifies the machine image used to launch EC2.
`instance_type = "t3.micro"`	Selects the EC2 instance type.

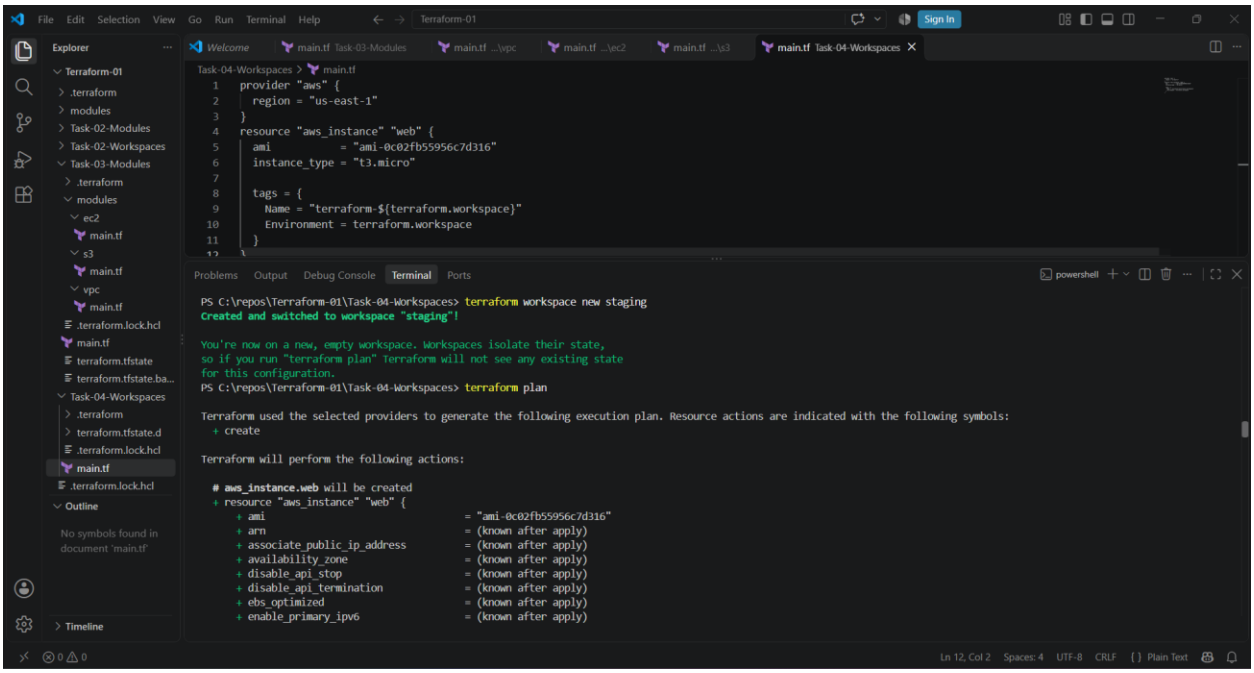
`terraform.workspace`	Returns the currently selected workspace name.
`"terraform-${terraform.workspace}"`	Creates environment-specific names such as `terraform-dev`.
`Environment = terraform.workspace`	Adds the environment name as an EC2 tag.

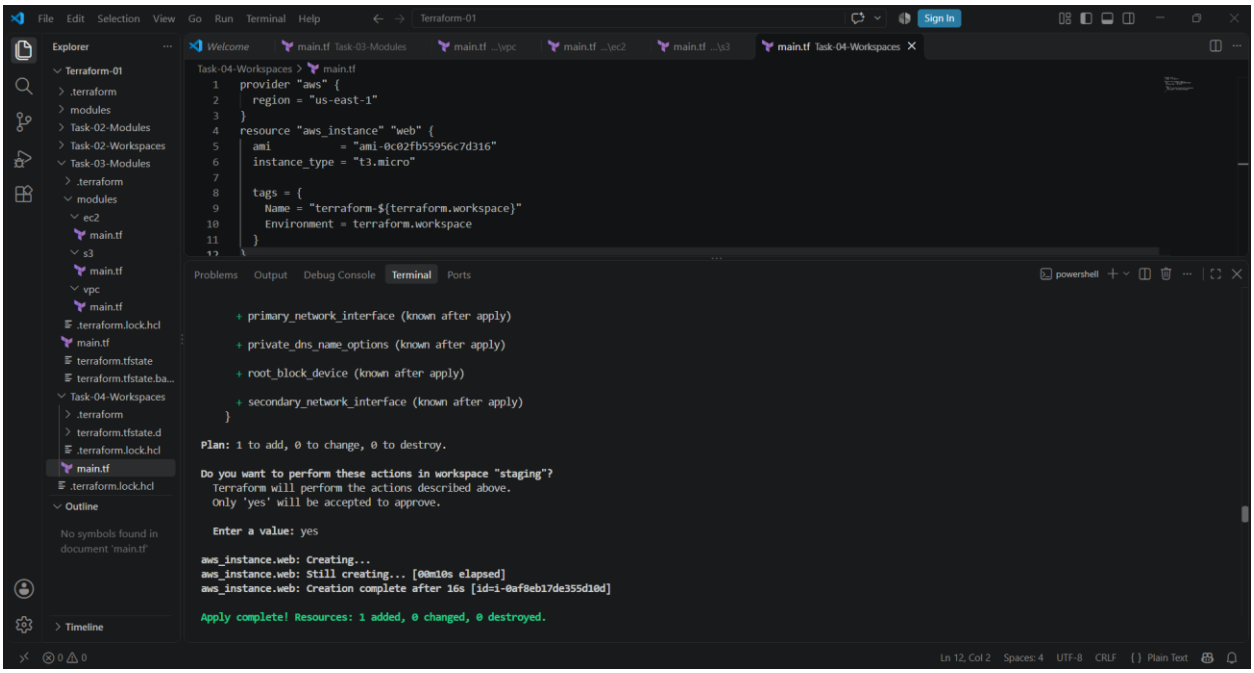
Example: When the active workspace is `dev`, Terraform creates an EC2 instance tagged `terraform-dev`. When the active workspace is `prod`, it creates `terraform-prod`.

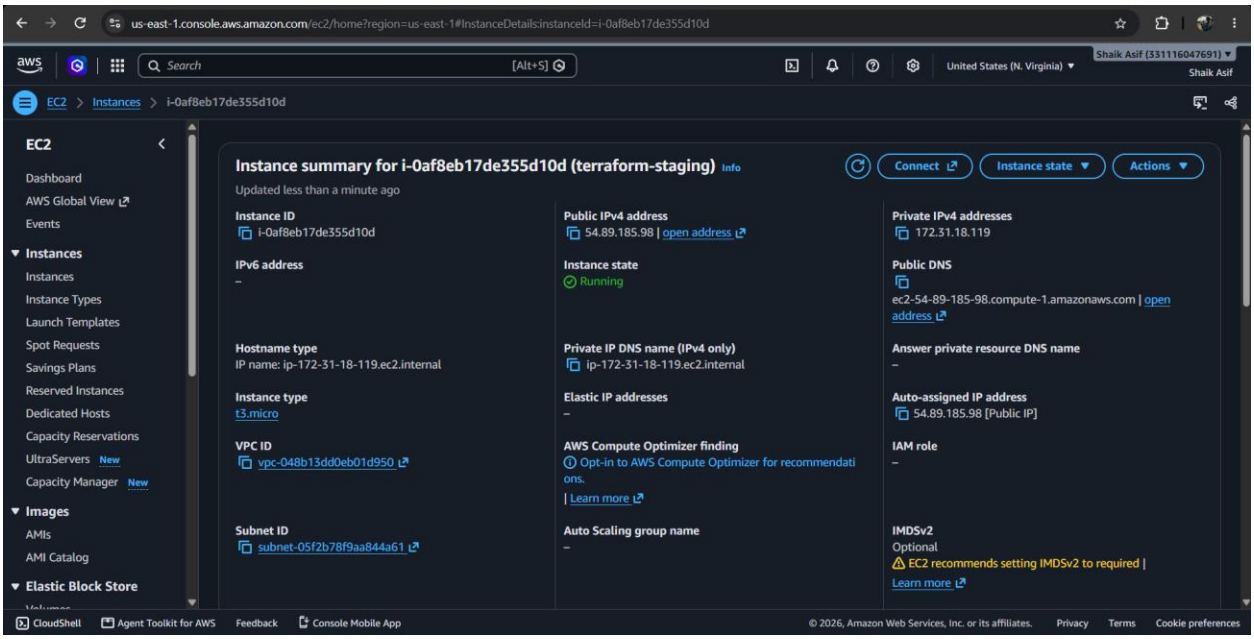


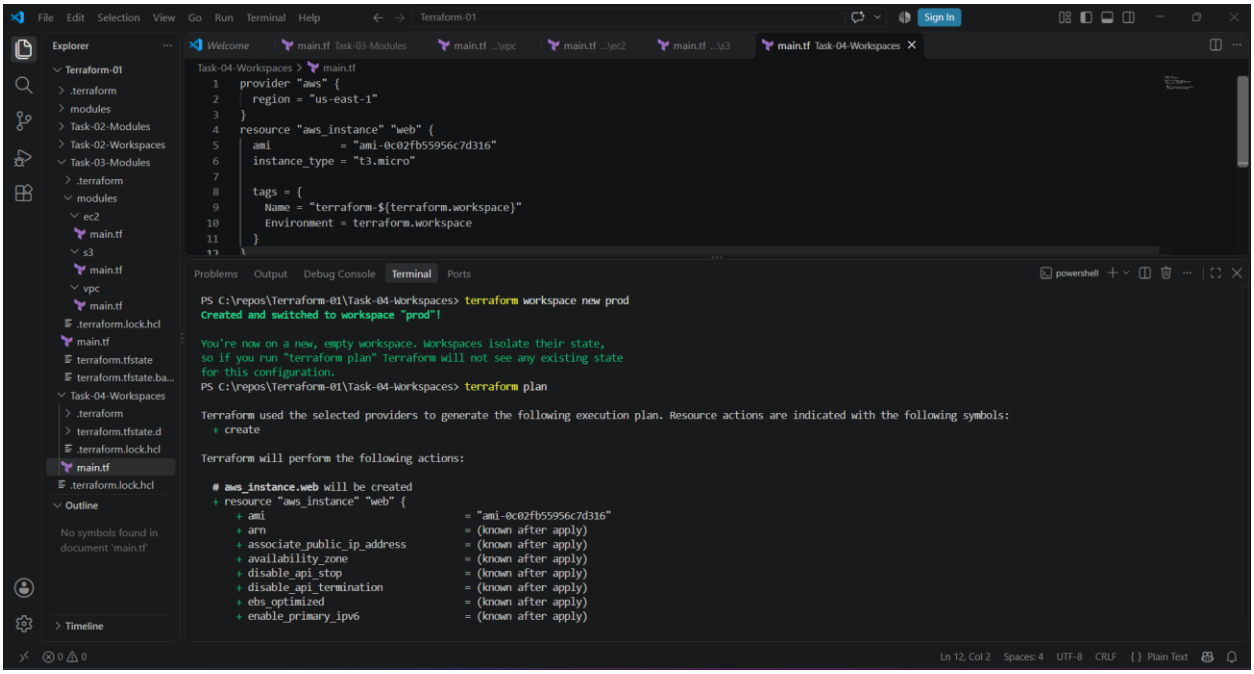

File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
.terraform
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.ba...
Task-04-Workspaces
.terraform
terraform.tfstate.d
.terraform.lock.hcl
main.tf
.terraform.lock.hcl
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
main.tf Task-04-Workspaces
Task-04-Workspaces > main.tf
1 provider "aws" {
2 region = "us-east-1"
3 }
4 resource "aws_instance" "web" {
5 ami = "ami-0c02fb55956c7d316"
6 instance_type = "t3.micro"
7
8 tags = {
9 Name = "terraform-${terraform.workspace}"
10 Environment = terraform.workspace
11 }
Problems Output Debug Console Terminal Ports
powershell
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions in workspace "dev"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m10s elapsed]
aws_instance.web: Creation complete after 17s [id=i-09fe441924a1a600e]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Ln 12, Col 2 Spaces: 4 UTF-8 CRLF { } Plain Text

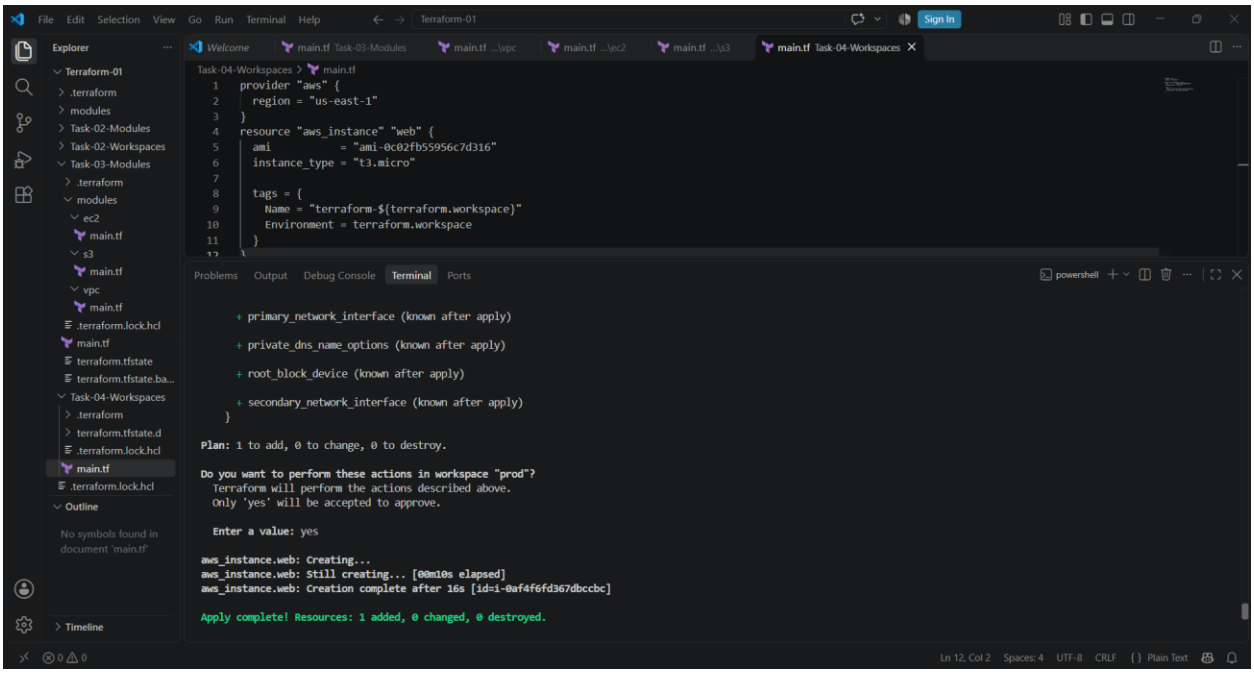

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-09fe441924a1a600e
Search [Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
EC2 > Instances > i-09fe441924a1a600e
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Instance summary for i-09fe441924a1a600e (terraform-dev) Info
Updated less than a minute ago
Connect
Instance state
Actions
Instance ID
i-09fe441924a1a600e
Public IPv4 address
3.84.180.19 | open address
Private IPv4 addresses
172.31.29.198
IPv6 address
–
Instance state
Running
Public DNS
ec2-3-84-180-19.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-172-31-29-198.ec2.internal
Private IP DNS name (IPv4 only)
ip-172-31-29-198.ec2.internal
Answer private resource DNS name
–
Instance type
t3.micro
Elastic IP addresses
–
Auto-assigned IP address
3.84.180.19 [Public IP]
VPC ID
vpc-048b13dd0eb01d950
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
| Learn more
IAM role
–
Subnet ID
subnet-05f2b78f9aa844a61
Auto Scaling group name
–
IMDSv2
Optional
EC2 recommends setting IMDSv2 to required | Learn more
Instance ARN
Managed
false
Operator
–
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

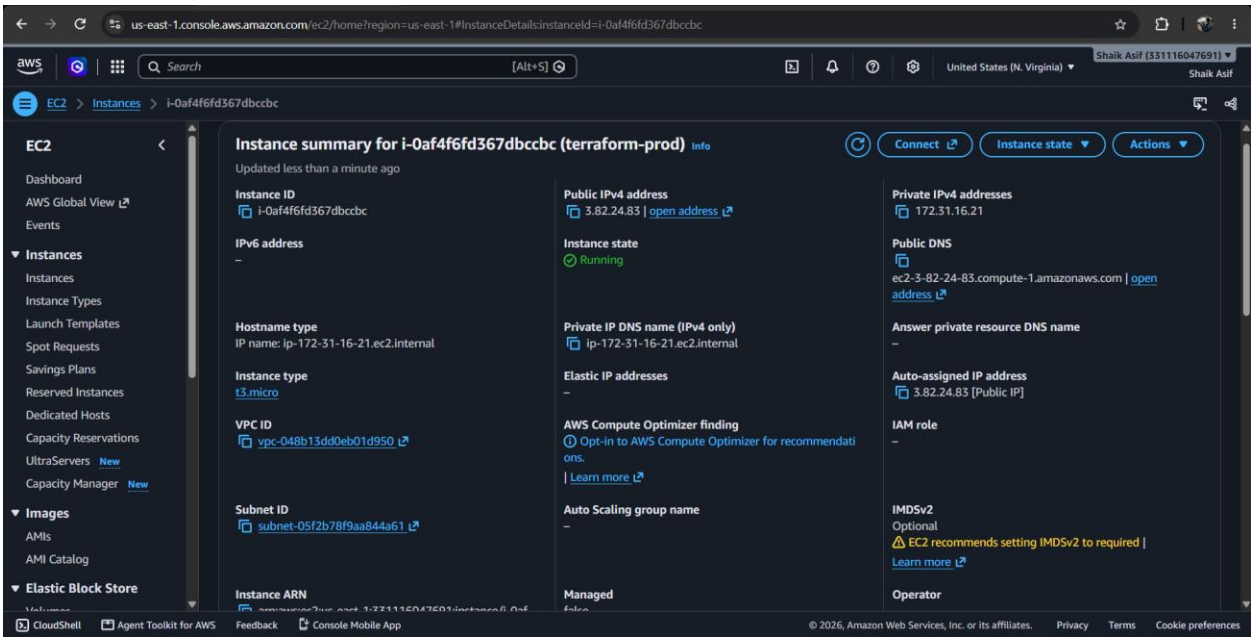

File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
.terraform
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.ba...
Task-04-Workspaces
.terraform
terraform.tfstate.d
.terraform.lock.hcl
main.tf
.terraform.lock.hcl
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
main.tf Task-04-Workspaces
Task-04-Workspaces > main.tf
1 provider "aws" {
2 region = "us-east-1"
3 }
4 resource "aws_instance" "web" {
5 ami = "ami-0c02fb55956c7d316"
6 instance_type = "t3.micro"
7
8 tags = {
9 Name = "terraform-${terraform.workspace}"
10 Environment = terraform.workspace
11 }
Problems Output Debug Console Terminal Ports
powershell
PS C:\repos\Terraform-01\Task-04-Workspaces> terraform workspace new staging
Created and switched to workspace "staging"!
You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\repos\Terraform-01\Task-04-Workspaces> terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
aws_instance.web will be created
+ resource "aws_instance" "web" {
+ ami = "ami-0c02fb55956c7d316"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
Ln 12, Col 2 Spaces: 4 UTF-8 CRLF { } Plain Text

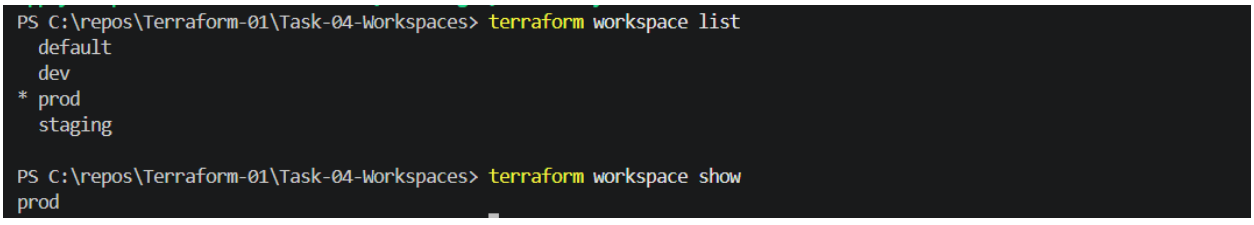

File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
.terraform
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.ba...
Task-04-Workspaces
.terraform
terraform.tfstate.d
.terraform.lock.hcl
main.tf
.terraform.lock.hcl
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
main.tf Task-04-Workspaces
Task-04-Workspaces > main.tf
1 provider "aws" {
2 region = "us-east-1"
3 }
4 resource "aws_instance" "web" {
5 ami = "ami-0c02fb55956c7d316"
6 instance_type = "t3.micro"
7
8 tags = {
9 Name = "terraform-${terraform.workspace}"
10 Environment = terraform.workspace
11 }
Problems Output Debug Console Terminal Ports
powershell
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions in workspace "staging"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m10s elapsed]
aws_instance.web: Creation complete after 16s [id=i-0af8eb17de355d10d]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Ln 12, Col 2 Spaces: 4 UTF-8 CRLF { } Plain Text


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0af8eb17de355d10d
Search
[Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
EC2 > Instances > i-0af8eb17de355d10d
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Instance summary for i-0af8eb17de355d10d (terraform-staging) Info
Updated less than a minute ago
Connect
Instance state
Actions
Instance ID
i-0af8eb17de355d10d
IPv6 address
–
Hostname type
IP name: ip-172-31-18-119.ec2.internal
Instance type
t3.micro
VPC ID
vpc-048b13dd0eb01d950
Subnet ID
subnet-05f2b78f9aa844a61
Public IPv4 address
54.89.185.98 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-172-31-18-119.ec2.internal
Elastic IP addresses
–
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
| Learn more
Auto Scaling group name
–
Private IPv4 addresses
172.31.18.119
Public DNS
ec2-54-89-185-98.compute-1.amazonaws.com | open address
Answer private resource DNS name
–
Auto-assigned IP address
54.89.185.98 [Public IP]
IAM role
–
IMDSv2
Optional
EC2 recommends setting IMDSv2 to required |
Learn more
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
modules
Task-02-Modules
Task-02-Workspaces
Task-03-Modules
.terraform
modules
ec2
main.tf
s3
main.tf
vpc
main.tf
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.ba...
Task-04-Workspaces
.terraform
terraform.tfstate.d
.terraform.lock.hcl
main.tf
.terraform.lock.hcl
Outline
No symbols found in document 'main.tf'
Timeline
Welcome
main.tf Task-03-Modules
main.tf ...\vpc
main.tf ...\ec2
main.tf ...\s3
main.tf Task-04-Workspaces
Task-04-Workspaces > main.tf
provider "aws" {
region = "us-east-1"
}
resource "aws_instance" "web" {
ami = "ami-0c02fb55956c7d316"
instance_type = "t3.micro"
tags = {
Name = "terraform-${terraform.workspace}"
Environment = terraform.workspace
}
Problems Output Debug Console Terminal Ports
powershell
PS C:\repos\Terraform-01\Task-04-Workspaces> terraform workspace new prod
Created and switched to workspace "prod"!
You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\repos\Terraform-01\Task-04-Workspaces> terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
aws_instance.web will be created
+ resource "aws_instance" "web" {
+ ami = "ami-0c02fb55956c7d316"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
Ln 12, Col 2
Spaces: 4
UTF-8
CRLF
Plain Text


Terraform-01
Task-04-Workspaces
main.tf
provider "aws" {
region = "us-east-1"
}
resource "aws_instance" "web" {
ami = "ami-0c02fb55956c7d316"
instance_type = "t3.micro"
tags = {
Name = "terraform-${terraform.workspace}"
Environment = terraform.workspace
}
}
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions in workspace "prod"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m10s elapsed]
aws_instance.web: Creation complete after 16s [id=i-0af4f6fd367dbccbc]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0af4f6fd367dbccbc
EC2 > Instances > i-0af4f6fd367dbccbc
Instance summary for i-0af4f6fd367dbccbc (terraform-prod)
Updated less than a minute ago
Connect
Instance state
Actions
Instance ID
i-0af4f6fd367dbccbc
Public IPv4 address
3.82.24.83 | open address
Private IPv4 addresses
172.31.16.21
IPv6 address
–
Instance state
Running
Public DNS
ec2-3-82-24-83.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-172-31-16-21.ec2.internal
Private IP DNS name (IPv4 only)
ip-172-31-16-21.ec2.internal
Answer private resource DNS name
–
Instance type
t3.micro
Elastic IP addresses
–
Auto-assigned IP address
3.82.24.83 [Public IP]
VPC ID
vpc-048b13dd0eb01d950
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more
IAM role
–
Subnet ID
subnet-05f2b78f9aa844a61
Auto Scaling group name
–
IMDSv2
Optional
EC2 recommends setting IMDSv2 to required
Learn more
Instance ARN
Managed
Operator


PS C:\repos\Terraform-01\Task-04-Workspaces> terraform workspace list
default
dev
* prod
staging
PS C:\repos\Terraform-01\Task-04-Workspaces> terraform workspace show
prod

6. Observation

- Three workspaces were created: `dev`, `staging`, and `prod`.
- One EC2 instance was provisioned in each workspace.
- All instances used the same Terraform configuration.
- The EC2 names and environment tags changed according to the selected workspace.
- Each workspace maintained its own Terraform state.

7. Conclusion

Successfully provisioned separate EC2 instances for Dev, Staging, and Prod using Terraform workspaces. This demonstrated how a single Terraform configuration can manage multiple environments while maintaining separate state files and reducing code duplication.

Interview answer: Terraform workspaces allow us to manage multiple environments using the same configuration while maintaining separate state files for each workspace.